LESSON 8

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Shift Register Magic

Control 8 LEDs with only 3 wires, and learn to count like a computer.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How a **shift register** (74HC595) gives you extra outputs
- How computers count in **binary** with only 0 and 1
- What a **bit** and a **byte** are
- How to store a light show in an **array** of patterns

SESSION PLAN

0:00	What is a shift register? What is binary?
0:15	Wire the chip and 8 LEDs
0:50	Sketch 1: **L08_OneByOne**
0:55	Short break
1:00	Sketch 2: **L08_Binary** (count like a computer)
1:20	Sketch 3: **L08_LightShow**
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, on your 2 breadboards
- ☐ 74HC595 chip (16 legs)
- ☐ 8 LEDs (any colours)
- ☐ 8 resistors, 220 Ω (red-red-brown)
- ☐ 10 kΩ potentiometer (the knob)
- ☐ About 20 jumper wires

1. Running out of pins?

Last lesson, one small display used up **8 pins**. What if you want 16 LEDs, or 32? The ESP32 would quickly run out of pins.

The **74HC595 shift register** is a little chip that fixes this. The ESP32 sends it 8 on/off values **one after another** down a single data wire. The chip collects them, then switches its 8 outputs all at once. That's **3 wires in, 8 outputs out**.

Big idea: three wires

- **DATA**: the next value, on (1) or off (0).
- **CLOCK**: a "tick" that says *take the next value now*. Each tick pushes (shifts) all the values one place along.
- **LATCH**: "Done! Show all 8 values on the outputs now."

2. Binary: counting with only 0 and 1

We count with ten digits (0–9). Computers only have **off** and **on**, so they count with two digits: **0** and **1**. This is called **binary**. One 0-or-1 is called a **bit**, and 8 bits make a **byte**.

Each place in a binary number is worth double the place to its right:

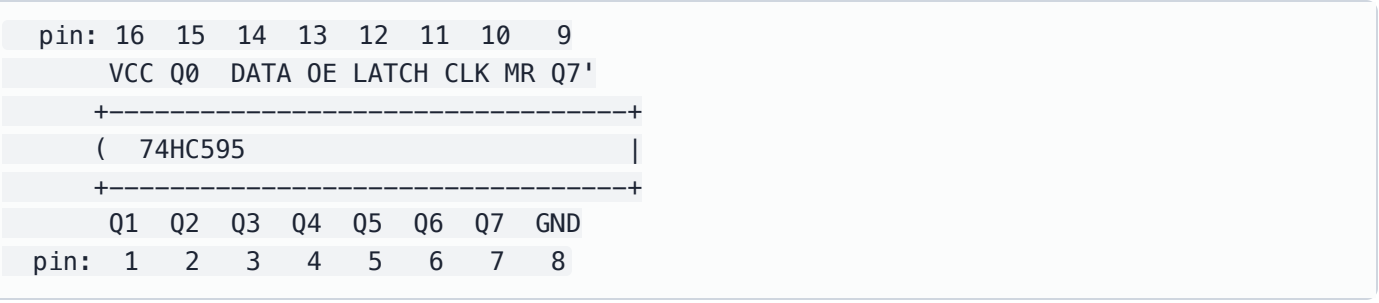
Place value	128	64	32	16	8	4	2	1
5 in binary	0	0	0	0	0	1	0	1
12 in binary	0	0	0	0	1	1	0	0
255 in binary	1	1	1	1	1	1	1	1

5 = 4 + 1, and 12 = 8 + 4. In code we write binary with **0b** in front: `0b00000101` is 5.

> **Try it on paper**
>
> Write **your age** as 8 bits:
>
> _____

3. Wire it up

The chip has a little **half-moon notch** at one end. Push the chip across the middle gap of a breadboard with the **notch pointing left**. Then pin 1 is bottom-left:



First give the chip power and connect it to the ESP32:

✓	From	To
☐	ESP32 **3V3**	**+** rail
☐	ESP32 **GND**	**–** rail
☐	chip pin **16** (VCC)	**+** rail
☐	chip pin **10** (MR)	**+** rail
☐	chip pin **8** (GND)	**–** rail
☐	chip pin **13** (OE)	**–** rail
☐	chip pin **14** (DATA)	ESP32 pin **23**
☐	chip pin **12** (LATCH)	ESP32 pin **19**
☐	chip pin **11** (CLOCK)	ESP32 pin **18**

Now put the 8 LEDs in a line on the breadboard. Each LED's **short leg** goes to a **220 Ω resistor**, and the resistor goes to the **–** rail. Then connect each **long leg** to a chip output. Put **Q7 on the left** and **Q0 on the right**, so the LEDs read like a binary number.

✓	LED (left → right)	long leg goes to chip pin
☐	LED 1 (128)	pin **7** (Q7)
☐	LED 2 (64)	pin **6** (Q6)
☐	LED 3 (32)	pin **5** (Q5)
☐	LED 4 (16)	pin **4** (Q4)
☐	LED 5 (8)	pin **3** (Q3)
☐	LED 6 (4)	pin **2** (Q2)
☐	LED 7 (2)	pin **1** (Q1)
☐	LED 8 (1)	pin **15** (Q0)

Last, add the **potentiometer** (the knob). We'll use it in Sketch 3:

✓	From	To
☐	potentiometer left leg	**+** rail (3V3)
☐	potentiometer middle leg	ESP32 pin **34**
☐	potentiometer right leg	**–** rail (GND)

> **Careful with the chip**
>
> Don't connect pin 16 to 5V: use **3V3**. If the chip gets hot, unplug the USB right away and check the power wires.

4. Sketch 1: One LED at a time

Type this in and save it as **L08_OneByOne**. You should see a light walk from right to left, then the LEDs flash every other one.

> **Lights in the wrong order?**
>
> If the light walks the wrong way or jumps around, check the LED wiring table again. If nothing lights, check pins 10 and 16 go to **+** and pins 8 and 13 go to **–**.

Type this in → save it as **L08_OneByOne** 31 lines

```
// Lesson 8: One LED at a time
// 74HC595 shift register: DATA 23, LATCH 19, CLOCK 18.

const int DATA_PIN = 23;
const int LATCH_PIN = 19;
const int CLOCK_PIN = 18;

void showByte(byte leds) {
  digitalWrite(LATCH_PIN, LOW);
  shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, leds);
  digitalWrite(LATCH_PIN, HIGH);
}
```

```
void setup() {
  pinMode(DATA_PIN, OUTPUT);
  pinMode(LATCH_PIN, OUTPUT);
  pinMode(CLOCK_PIN, OUTPUT);
}

void loop() {
  for (int i = 0; i < 8; i++) {
    showByte(bit(i));
    delay(200);
  }
  for (int i = 0; i < 4; i++) {
    showByte(0b10101010);
    delay(300);
    showByte(0b01010101);
    delay(300);
  }
}
```

Code	What it means
`byte leds`	A **byte** is a number from 0 to 255: exactly 8 bits, one for each LED
`digitalWrite(LATCH_PIN, LOW);`	"Get ready, I'm sending new values"
`shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, leds);`	Send the 8 bits one at a time, ticking the clock for each. **MSBFIRST** means the left-most bit goes first
`digitalWrite(LATCH_PIN, HIGH);`	"Show them now!" All 8 LEDs change at once
`bit(i)`	A byte with only bit **i** switched on. `bit(3)` is `0b00001000`
`0b10101010`	A binary number: every other LED is on

5. Sketch 2: Count in binary

Save a copy as **L08_Binary** and change it to match this. Open the Serial Monitor too.

Type this in → save it as **L08_Binary** — 29 lines

```
// Lesson 8: Count in binary
// The 8 LEDs show the number in binary. Watch the Serial Monitor too!

const int DATA_PIN = 23;
const int LATCH_PIN = 19;
const int CLOCK_PIN = 18;

void showByte(byte leds) {
  digitalWrite(LATCH_PIN, LOW);
  shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, leds);
  digitalWrite(LATCH_PIN, HIGH);
```

```
}

void setup() {
  Serial.begin(115200);
  pinMode(DATA_PIN, OUTPUT);
  pinMode(LATCH_PIN, OUTPUT);
  pinMode(CLOCK_PIN, OUTPUT);
}

void loop() {
  for (int n = 0; n < 256; n++) {
    showByte(n);
    Serial.print(n);
    Serial.print(" in binary is ");
    Serial.println(n, BIN);
    delay(500);
  }
}
```

Code	What it means
`showByte(n);`	Show the number **n** on the LEDs as binary
`Serial.println(n, BIN);`	Print **n** in binary instead of normal numbers

Try it

1. Watch the right-most LED. How often does it change?
2. When does the **left-most** LED switch on for the first time? Which number is it?

---

3. Change `delay(500)` to `delay(20)`. How long does it take to count to 255 now?

6. Sketch 3: Light show

Now we'll store a whole light show in an **array** of patterns, and use the knob to change the speed. Save a copy as **L08_LightShow** and change it to match:

Type this in → save it as **L08_LightShow** — 39 lines

```
// Lesson 8: Light show
// Plays a list of patterns. The knob on pin 34 sets the speed.

const int DATA_PIN = 23;
const int LATCH_PIN = 19;
const int CLOCK_PIN = 18;
const int POT_PIN = 34;

const int NUM_PATTERNS = 8;
const byte patterns[NUM_PATTERNS] = {
  0b10000001,
```

```
  0b01000010,
  0b00100100,
  0b00011000,
  0b00011000,
  0b00100100,
  0b01000010,
  0b10000001
};

void showByte(byte leds) {
  digitalWrite(LATCH_PIN, LOW);
  shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, leds);
  digitalWrite(LATCH_PIN, HIGH);
}

void setup() {
  pinMode(DATA_PIN, OUTPUT);
  pinMode(LATCH_PIN, OUTPUT);
  pinMode(CLOCK_PIN, OUTPUT);
}

void loop() {
  for (int i = 0; i < NUM_PATTERNS; i++) {
    showByte(patterns[i]);
    int wait = map(analogRead(POT_PIN), 0, 4095, 30, 500);
    delay(wait);
  }
}
```

Code	What it means
`const byte patterns[NUM_PATTERNS] = {…};`	A list of 8 patterns. Each line is one "frame" of the show
`map(analogRead(POT_PIN), 0, 4095, 30, 500)`	Turn the knob reading (0–4095) into a wait of 30–500 ms

Design on paper first

Draw 8 circles for each frame, colour in the ones that should be on, then write the 1s and 0s.

7. Challenges

Challenge 1: Your own show

Change the patterns to make your own light show. Try to make it at least 12 frames long. Remember to change `NUM_PATTERNS`!

Challenge 2: Knight Rider

Make one light sweep from right to left and back again, like a scanner. *Hint: use two* `for` *loops, one going up and one going down, with* `bit(i)`.

Challenge 3: Bar graph

Make a meter. The further you turn the knob, the more LEDs light up, from 0 to 8. *Hint:* `map(…, 0, 4095, 0, 8)` *tells you how many to light.*

Bonus: Binary quiz

Show a random number from 1 to 15 on the LEDs. Your friend has to say the number in normal counting!

8. Quiz

Check what you learned

1. What do the three wires DATA, CLOCK and LATCH do?

   _____

   _____

2. Write the number **10** in binary (8 bits).

   _____

3. What is the biggest number one byte can hold?

   _____

Tidy up: unplug the USB cable. Gently lever the chip out with a flat tool from both ends so you don't bend its legs.